

# PROJECT REPORT

## Trekking Management System V2

---

### Student Details

- **Student Name:** Atul Pandey
  - **Roll Number:** 22f2000145
  - **Course:** IITM BS Degree in Data Science and Applications
  - **Project Term:** May-2026
  - **Submission Date:** July 10, 2026
- 

### Project Overview

#### Problem Statement

Adventure and trekking organizations require an efficient software system to manage trek organizers, field guides, and participants. Traditional methods such as spreadsheets, phone calls, and messaging platforms often result in operational inefficiencies, overbooking, poor coordination, and lack of booking status tracking.

The **Trekking Management System V2 (TMS)** addresses these challenges by providing a secure role-based web application with dedicated workflows for three different user roles.

#### Admin (Superuser)

The administrator is responsible for:

- Managing trek routes
- Adding staff members
- Approving or rejecting guide registrations
- Monitoring bookings
- Confirming payments
- Cancelling bookings when necessary
- Viewing overall system statistics

## **Trek Staff (Guide)**

The trekking guide can:

- View assigned treks
- Update trek status (Open, Closed, Completed)
- Manage available participant slots
- View the list of registered trekkers

## **Trekker (User)**

A registered user can:

- Create an account
  - Log in securely
  - Update profile information
  - Search and filter available treks
  - Book treks
  - Make simulated payments
  - Track booking status
- 

# **Technical Architecture**

The project follows a modular architecture that clearly separates frontend and backend responsibilities.

## **Backend (Flask + SQLite)**

The backend is developed using Flask REST APIs.

Its responsibilities include:

- Processing client requests
- Performing CRUD operations
- Business logic validation
- Preventing overbooking
- Allowing bookings only for Open treks
- Role-based authentication using Flask-Security-Too

SQLite serves as the application's relational database.

## **Frontend (Vue.js + Bootstrap)**

The frontend is implemented as a Single Page Application (SPA) using:

- Vue.js
- Vue Router
- Bootstrap 5

The application is responsive and works across desktop and mobile devices without requiring a complex build process.

## **Background Processing (Celery + Redis)**

Redis is used both as:

- An in-memory cache
- A Celery message broker

Celery performs long-running asynchronous tasks, including:

- Sending Google Chat notifications
- Monthly activity report generation
- Background CSV export

This architecture ensures that lengthy operations do not block the user interface.

---

# **AI / LLM Declaration**

As per the course guidelines, AI was used as a development assistant.

## **AI Usage**

The AI assistant helped with:

- Testing the api endpoint
- Finding the bug and resolving it
- Fixing the frontend
- Fixing issue of backend jobs

All generated suggestions were manually reviewed, modified where necessary, tested locally, and validated before being incorporated into the final project.

In general total ai usage was in between 20%-25%

---

# Section 4: Frameworks and Libraries

## Backend

- Flask 3.1.3
- Flask-RESTful 0.3.10
- Flask-SQLAlchemy 3.1.1
- Flask-Security-Too 5.8.1

### Purpose

- REST API development
  - Database ORM
  - Authentication
  - Role-Based Access Control (RBAC)
- 

## Caching and Background Tasks

- Redis 6.4.0
- Flask-Caching 2.3.0
- Celery 5.6.3

### Purpose

- API caching
  - Message brokering
  - Background task execution
  - Periodic scheduling
- 

## Frontend

- Vue.js (CDN)
- Vue Router
- Bootstrap 5.3.3

### Purpose

- Reactive frontend

- Client-side routing
  - Responsive user interface
- 

## Section 5: Database Design

The application uses five relational database tables.

### Database Entities

- Role
- User
- User\_Roles
- Trek
- Booking

### Relationship Summary

#### Users ↔ Roles

- Many-to-Many relationship through the **user\_roles** table.

#### Guide ↔ Trek

- One guide can manage multiple treks.
- Each trek has one assigned guide.

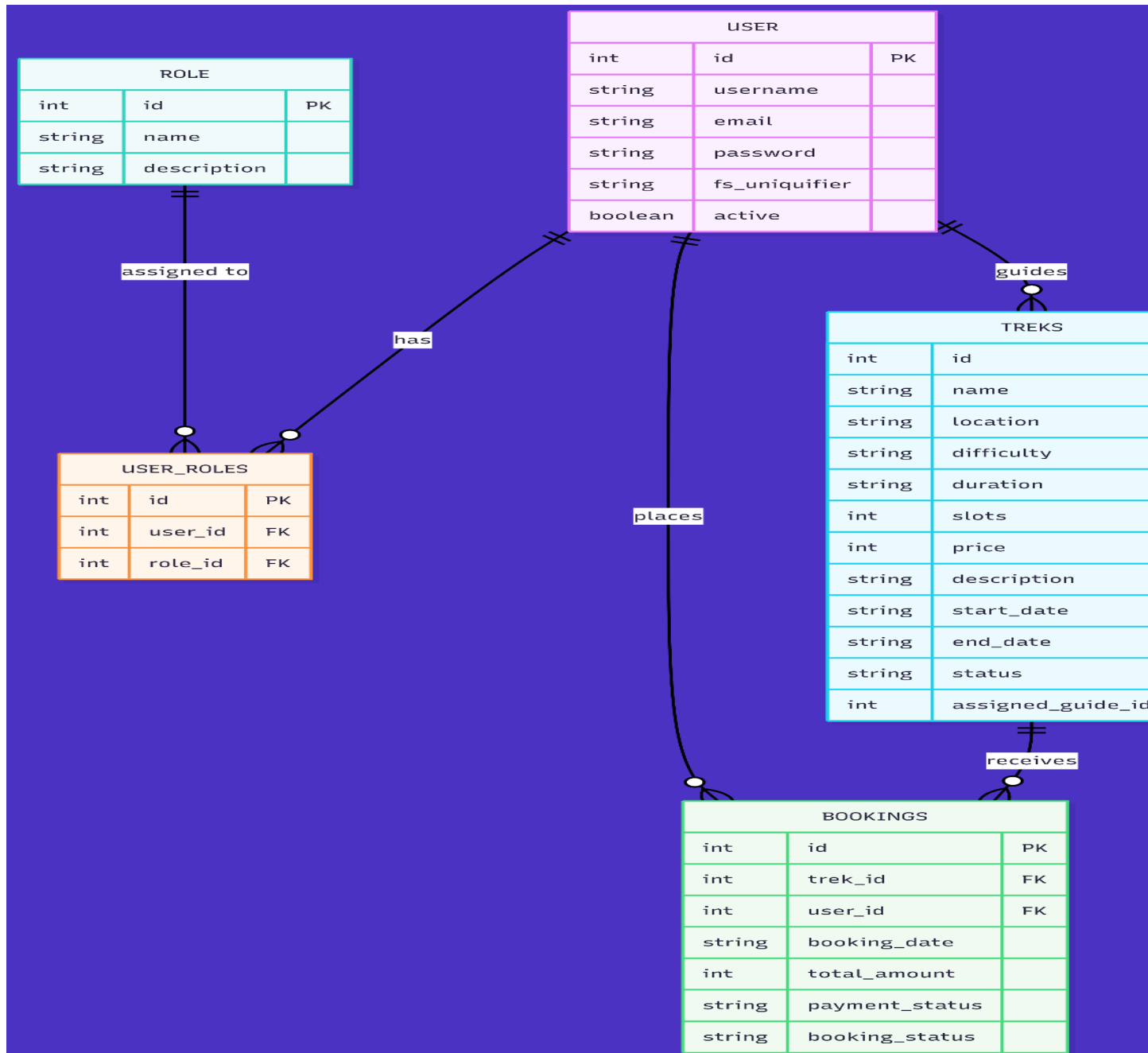
#### User ↔ Booking

- One user can create multiple bookings.

#### Trek ↔ Booking

- One trek can have multiple bookings.
- Deleting a trek automatically removes related bookings.

### ER Diagram



## Section 6: REST API Endpoints

The application uses token-based authentication through the **Authentication-Token** header.

| Method | Endpoint | Roles | Purpose |
|--------|----------|-------|---------|
|--------|----------|-------|---------|

|           |                               |                |                          |
|-----------|-------------------------------|----------------|--------------------------|
| POST      | /api/login                    | Public         | User login               |
| POST      | /api/register                 | Public         | User registration        |
| GET       | /api/admin/pending-staff      | Admin          | View pending staff       |
| POST      | /api/admin/approve-staff/{id} | Admin          | Approve staff            |
| POST      | /api/admin/reject-staff/{id}  | Admin          | Reject staff             |
| GET       | /api/admin/stats              | Admin          | Dashboard statistics     |
| GET       | /api/admin/guides             | Admin          | View guides              |
| GET       | /api/admin/users              | Admin          | View users               |
| POST      | /api/admin/create-staff       | Admin          | Create guide             |
| POST      | /api/admin/toggle-user/{id}   | Admin          | Activate/Deactivate user |
| GET       | /api/treks                    | All Roles      | Retrieve treks           |
| POST      | /api/treks/create             | Admin          | Create trek              |
| PUT       | /api/treks/update/{id}        | Admin, Staff   | Update trek              |
| DELETE    | /api/treks/delete/{id}        | Admin          | Delete trek              |
| GET       | /api/bookings                 | All Roles      | Retrieve bookings        |
| POST      | /api/bookings/create          | User           | Create booking           |
| PUT       | /api/bookings/update/{id}     | User, Admin    | Update booking           |
| GET       | /api/export                   | All Roles      | Background CSV export    |
| GET / PUT | /api/user/profile             | Logged-in User | Manage profile           |

---

## Section 7: Project Demonstration

### Presentation Video

Google Drive Link

[VIDEO](#)

## Demonstration Includes

- Project introduction
  - User registration and login
  - Trek search and filtering
  - Booking workflow
  - Payment simulation
  - CSV export
  - Staff operations
  - Admin dashboard
  - Statistics and payment approval.
- 

## Conclusion

The **Trekking Management Application V2** successfully demonstrates the implementation of a modern full-stack web application using the prescribed technology stack. The project integrates secure authentication, role-based access control, RESTful APIs, asynchronous task execution, caching, responsive frontend design, and relational database management into a cohesive solution.

The application effectively automates trekking operations while improving scalability, maintainability, and user experience through background processing and modular system architecture.